

Name: Adalynn Cadence Everly

ID: 1001852753

Class: CSE-4321-001

HW 2:

Section 6.1: Problem 3 Section 6.2: problem 4. Here is the link to the HW2 question descriptions: [HW2_questions.pdf](#)

3. Answer the following questions for the method `search()` below:

```
public static int search (List list, Object element)
// Effects: if list or element is null throw NullPointerException
//   else if element is in the list, return an index
//   of element in the list; else return -1
//   for example, search ([3,3,1], 3) = either 0 or 1
//       search ([1,7,5], 2) = -1
```

Base your answer on the following characteristic partitioning:

```
Characteristic: Location of element in list
    Block 1: element is first entry in list
    Block 2: element is last entry in list
    Block 3: element is in some position other than first or last
```

- (a) “Location of element in list” fails the disjointness property. Give an example that illustrates this.
- The partition “Location of element in list” fails disjointness because the same value can land in more than one block at once. For example, if you call `search([4, 2, 4], 4)`, the number 4 appears at index 0 and also at index 2. That puts it in both Block 1 (first) and Block 2 (last), depending on which copy the method finds first. So the blocks aren’t mutually exclusive.
- (b) “Location of element in list” fails the completeness property. Give an example that illustrates this.
- It also fails completeness. The list `[7, 8, 9]` with `element = 5` returns -1 because 5 isn’t in the list. But the current blocks only talk about positions when the element is actually in the list. They don’t account for the case where the element isn’t there at all, which is still a valid outcome, so the partition is incomplete.
- (c) Supply one or more new partitions that capture the intent of “Location of element in list” but do not suffer from completeness or disjointness problems.
- A better set of blocks that covers everything and avoids overlap would be:
 - Block (`␣`): element...(appears)
 - A: not in the list

- B: 1 time & it's the 1st item
- C: 1 time & it's the last item
- D: 1 time & it's somewhere in middle
- E: multiple times
- This set covers every possible case and keeps each value in exactly one block.

4. Answer the following questions for the method `intersection()` below:

```
public Set intersection (Set s1, Set s2)
    // Effects:   If s1 or s2 is null throw NullPointerException
    //           else return a (non null) Set equal to the intersection
    //           of Sets s1 and s2

    Characteristic: Validity of s1
        - s1 = null
        - s1 = {}
        - s1 has at least one element

    Characteristic: Relation between s1 and s2
        - s1 and s2 represent the same set
        - s1 is a subset of s2
        - s2 is a subset of s1
        - s1 and s2 do not have any elements in common
```

- (a) Does the partition “Validity of s1” satisfy the completeness property? If not, give a value for s1 that does not fit in any block.
- No, the partition "Validity of s1" doesn't satisfy completeness. The blocks are: `s1 == null`, `s1 == {}`, `s1 has at least one element`. But if s1 is a special subclass of Set, like a TreeSet that throws an exception in `equals()`, it technically doesn't fit clearly into any block based on current wording. Also, the partition assumes only 3 cases exist, which might miss cases where s1 is non-null but still behaves weirdly.
- (b) Does the partition “Validity of s1” satisfy the disjointness property? If not, give a value for s1 that fits in more than one block.
- A good example is when s1 is an empty set, like `s1 equals empty braces`. That obviously matches the block that says s1 is empty. But depending on how someone reads the block that says s1 has at least one element, they might confuse the two or think the empty set sort of fits both. The way those blocks are written, it's easy to mess up which one an input belongs in. You need each input to clearly belong to one and only one block, and that isn't happening here.
- (c) Does the partition “Relation between s1 and s2” satisfy the completeness property? If not, give a pair of values for s1 and s2 that does not fit in any block.
- Right now, it talks about s1 and s2 being equal, one being a subset of the other, or them having elements in common. But what about when s1 and s2 have absolutely nothing to do with each other? For example, if s1 is {1, 2} and s2 is {3, 4}, they're not equal, s1 isn't inside s2, and they don't overlap at all. That case doesn't fit into any of the current blocks. So the partition is incomplete because it skips a very real possibility.

- (d) Does the partition “Relation between s_1 and s_2 ” satisfy the disjointness property? If not, give a pair of values for s_1 and s_2 that fits in more than one block.
- Say s_1 is $\{1, 2\}$ and s_2 is $\{1, 2, 3\}$. s_1 fits as a subset of s_2 , but it also shares elements with s_2 . So it could fit into both blocks at the same time. That breaks the rule that says each input should only match one category. If your partition lets the same input fall into two different groups, it's not truly split up the right way.